



Gymnázium J. K. Tyla

Agentické inženýrství a návrh řídícího systému pro automatizovaný softwarový vývoj

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D
Vedoucí práce: Michal Dočekal

Rozsah práce: 942 slov / 6459 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

[cz] Tato odborná práce se zabývá principy agentního inženýrství (*agentic engineering*) a architekturou řídicích harnessů pro automatizovaný vývoj softwaru. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od sémantické analýzy požadavků v GitHub Issues, přes technické plánování, až po generování kódu a vystavení pull requestu. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely proti uvíznutí v nekonečných cyklech, správu kontextu bez sémantického posunu a především kontinuální zapojení člověka formou schvalovacích bran (*Human-in-the-loop*).

Klíčová slova

Agent, Agentní inženýrství, Chatbot, Dovednosti, Git, GitHub, Jazykový model (LLM) , Smyčka
ReAct (Agent Loop) , Vektorová reprezentace, Řídicí systém (Harness)

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	4
1 Úvod	9
1.1 Motivace a vymezení problému	9
1.2 Cíl práce a výzkumné otázky	9
1.3 Metodika práce	9
2 Teoretická část: Vymezení konceptu	10
2.1 Správa verzí [Version Control], Plánování [Planning], Kontinuální integrace [Continuous Integration] (CI a GitHub Actions) a Požadované kontroly [Required Checks]	10
2.1.1 Git a GitHub	10
2.1.2 Pull Request	10
2.2 Large Language Model (LLM) , chatboti a agenti	10
2.2.1 Úvod	10
2.2.2 Tokeny, tokenizace a Vektorová reprezentace [Embedding]	11
2.3 Agentní inženýrství a Control Harness (Harness)	11
2.3.1 Úvod	11
2.3.2 Agent vs. Chatbot	12
2.3.3 Dovednosti	12
3 Praktická část – Návrh architektury	13
3.1 Git a GitHub	13
4 Výsledky a diskuse	14
5 Závěr	14
Rejstřík	14
A	14
Agent	14
Agentní inženýrství	14
C	14
Chatbot	14
D	14
Degradace kontextu	14
Dovednosti	14
G	14

Generování rozšířené vyhledáváním (RAG)	14
Git	14
GitHub	15
GitHub Actions (Actions)	15
I	15
Inženýrství pracovních grafů (Graph Engineering)	15
Inženýrství prováděcí smyčky (Loop Engineering)	15
J	15
Jazykový model (LLM)	15
K	15
Kompakce kontextu (Compaction)	15
Kontextové inženýrství	15
Kontextové okno	15
M	16
Mezipaměť klíčů a hodnot (KV Cache)	16
Model Context Protocol (MCP)	16
O	16
Orientovaný acyklický graf (DAG)	16
P	16
Požadavek na sloučení	16
Promptové inženýrství	16
Průběžná integrace (CI)	16
R	16
Rozšíření	16
S	16
Skript	16
Sloučení commitů (Squash)	17
Sloučení větví (Merge)	17
Smyčka ReAct (Agent Loop)	17
Softwarové inženýrství	17
Softwarový kontejner (Container)	17
Správa verzí	17
T	17
Tah interakce (Turn)	17
Token	17
Tokenizér	17

Transformerová architektura (Transformer)	17
U	18
Událostní zachytný bod (Hook)	18
V	18
Vektorová reprezentace	18
Větev repozitáře (Branch)	18
Z	18
Zapojení člověka do smyčky (HITL)	18
Ř	18
Řídicí systém (Harness)	18
Seznam příloh	19
Seznam zdrojů	20

Seznam obrázků a tabulek



KAPITOLA 1

Úvod

1.1 Motivace a vymezení problému

1.2 Cíl práce a výzkumné otázky

Hlavní cíl: Vymezit teoretické principy agentního inženýrství (*agentic engineering*) a navrhnout modulární architekturu řídicího harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

Dílčí cíle:

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu řídicího harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Vzhledem k dynamickému vývoji v oblasti autonomního softwarového vývoje práce důsledně zachovává a integruje zavedené anglické odborné názvy (např. *harness*, *pull request*, *agent loop*, *prompt engineering*, *skills* či *context rot*). Použití této terminologie je integrální součástí práce, neboť tyto anglické pojmy představují de facto celosvětové průmyslové standardy (*industry standards*), jejichž doslovný český překlad by byl nejednoznačný, zavádějící či v rozporu s běžnou inženýrskou praxí.

Postup práce sleduje strukturu inženýrského cyklu:

- **1. Analýza konceptu:** Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- **2. Návrh architektury:** Formulace modulárního modelu řídicího harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- **3. Kritické zhodnocení:** Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.



KAPITOLA 2

Teoretická část: Vymezení konceptu

2.1 Správa verzí [Version Control], Plánování [Planning], Kontinuální integrace [Continuous Integration] (CI a GitHub Actions) a Požadované kontroly [Required Checks]

2.1.1 Git a GitHub

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém **Git** v kombinaci s platformou **GitHub**.

Klíčové komponenty infrastruktury zahrnují:

- **Distribuovaný systém Git** (1): Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- **Platforma GitHub**: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - **Zadávání a sledování úkolů (Issues)**: Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - **Revize změn (Pull Requests)**: Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - **Automatizace (GitHub Actions)**: Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Agent v tomto pojetí nevystupuje jako černá skříňka s proprietárním protokolem, nýbrž jako standardní přispěvatel, který plně respektuje běžné vývojářské zvyklosti a nástroje.

2.1.2 Pull Request

2.2 Large Language Model (LLM) , chatboti a agenti

2.2.1 Úvod

V agentním softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový transformer (*Decoder-only*), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek (2). Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- **Autoregresivní predikce**: Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).

- **Stochastická povaha:** Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném řídicím harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť.

2.2.2 Tokeny, tokenizace a Vektorová reprezentace [Embedding]

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- **Tokeny a tokenizér:** Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte Pair Encoding, BPE).
- **Vektorová reprezentace**★ — [CZ] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné sémantické vztahy mezi reprezentacemi. (např. vektorová analogie král — muž + žena $\approx$ královna).
- **Jazyková asymetrie tokenizace:** Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.3 Agentní inženýrství a Control Harness (Harness)

2.3.1 Úvod

V terminologii agentního inženýrství používá tato práce pojem **Řídicí systém (Harness)**

★. [CZ] Řídicí postroj — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely, práci se stavem a deterministické řízení životního cyklu požadavku.. Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; veškerou orchestraci, práci se soubory a řízení bezpečnosti zajišťuje harness.

Ústřední komponentou a hlavní prováděcí funkcí, která v architektuře harnessu řídí samotný běh a iterativní koordinaci agenta v reálném vývojovém prostředí, je **Smyčka ReAct (Agent Loop)** ★. [CZ] Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí..

2.3.2 Agent vs. Chatbot

Chatbot★ — [cz] Systém založený na jazykovém modelu určený primárně k textové interakci s uživatelem; odpovídá na jednotlivé požadavky, ale sám o sobě nedisponuje autonomní prováděcí smyčkou ani nástroji pro samostatnou modifikaci okolního prostředí.. **Agent**★ — [cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí víceukrokové akce směřující k dosažení zadaného inženýrského cíle.. Rozdíl mezi nimi nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí.

Srovnání obou přístupů:

- **Konverzační chatbot:**
 - Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
 - Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
 - Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- **Autonomní agent:**
 - Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
 - Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
 - Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

2.3.3 Dovednosti

Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží **Dovednosti**★ — [cz] Znovupoužitelné modulární balíčky instrukcí (typicky definovaných v souboru SKILL.md), procedurálních pravidel a volitelných pomocných skriptů či zdrojů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu..



KAPITOLA 3

Praktická část – Návrh architektury

3.1 Git a GitHub



KAPITOLA 4

Výsledky a diskuse



KAPITOLA 5

Závěr

Rejstřík

A

Agent

[cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí více krokové akce směřující k dosažení zadaného inženýrského cíle.

Agentní inženýrství

[cz] Inženýrská disciplína zaměřená na návrh, orchestraci a provoz agentních systémů kolem jazykových modelů, včetně nástrojů, kontextu, prováděcích smyček, bezpečnostních mantinelů a lidského dohledu.

C

Chatbot

[cz] Systém založený na jazykovém modelu určený primárně k textové interakci s uživatelem; odpovídá na jednotlivé požadavky, ale sám o sobě nedisponuje autonomní prováděcí smyčkou ani nástroji pro samostatnou modifikaci okolního prostředí.

D

Degradace kontextu

[cz] Degradace pozornosti a kvality logického uvažování modelu způsobená zaplněním kontextového okna dlouhou historií, šumem nebo vzájemně si konkurujícími informacemi, která vede k přehlížení instrukcí a ztrátě souvislostí.

Dovednosti

[cz] Znovupoužitelné modulární balíčky instrukcí (typicky definovaných v souboru SKILL.md), procedurálních pravidel a volitelných pomocných skriptů či zdrojů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu.

G

Generování rozšířené vyhledávání (RAG)

[cz] Architektura, v níž systém před generováním nebo během něj vyhledá relevantní informace z externího zdroje a vloží je do kontextu modelu, aby výstup mohl být založen na načtených datech.

Git

[cz] Distribuovaný systém správy verzí, který uchovává historii projektu, podporuje větvení a slučování změn a umožňuje deterministický návrat k předchozím stavům repozitáře.

GitHub

[cz] Cloudová platforma pro hosting gitových repozitářů, správu vývojového cyklu pomocí Issues a Pull Requests a automatizaci CI/CD pracovních postupů.

GitHub Actions (Actions)

[cz] Automatizační platforma GitHubu, která spouští deklarované workflow a jejich joby v reakci na události repozitáře nebo ruční spuštění.

I

Inženýrství pracovních grafů (Graph Engineering)

[cz] Návrh agentních nebo automatizačních pracovních postupů jako explicitních grafů uzlů, závislostí a přechodů namísto jediné neomezené smyčky.

Inženýrství prováděcí smyčky (Loop Engineering)

[cz] Návrh a řízení iterativní prováděcí smyčky agenta: stavových přechodů, podmínek ukončení, rozpočtů, opakování, eskalací a vazby mezi rozhodováním modelu a nástroji.

J

Jazykový model (LLM)

[cz] Velký jazykový model je neuronový model trénovaný nad rozsáhlými textovými daty, který autoregresivně zpracovává a generuje posloupnosti tokenů. V této práci vystupuje jako inferenční kognitivní jádro agentního systému.

K

Kompakce kontextu (Compaction)

[cz] Proces zmenšení aktivního kontextu, typicky shrnutím, výběrem nebo nahrazením starších částí historie kompaktnější reprezentací tak, aby se běh vešel do kontextového okna.

Kontextové inženýrství

[cz] Systematický návrh, výběr, pořadí a životní cyklus informací zpřístupňovaných modelu v aktivním kontextu, včetně instrukcí, paměti, nástrojových výsledků a externě načtených dat.

Kontextové okno

[cz] Maximální rozsah tokenové sekvence, kterou model při jednom běhu dokáže zahrnout do aktivního kontextu. Prakticky omezuje součet instrukcí, historie, nástrojových výstupů a dalších dat předávaných modelu.

M

Mezipaměť klíčů a hodnot (KV Cache)

[cz] Mezipaměť dříve vypočtených vektorů klíčů a hodnot v pozornostních vrstvách transformeru, která při autoregresivním generování omezuje nutnost opakovaně přepočítávat předchozí tokeny.

Model Context Protocol (MCP)

[cz] Model Context Protocol — otevřený standard původně navržený společností Anthropic pro standardizovanou komunikaci AI aplikací s externími nástroji, zdroji a daty prostřednictvím zpráv JSON-RPC.

O

Orientovaný acyklický graf (DAG)

[cz] Orientovaný graf bez orientovaného cyklu. V pracovních postupech umožňuje explicitně vyjádřit závislosti mezi kroky a pořadí, které z nich vyplývá.

P

Požadavek na sloučení

[cz] Formální návrh na začlenění změn z jedné větve repozitáře do druhé, který slouží jako místo pro automatizované kontroly, lidskou revizi a diskusi nad navrženými úpravami.

Promptové inženýrství

[cz] Inženýrská metodika systematického návrhu, strukturování a optimalizace instrukcí a systémových promptů pro řízení chování a mantinelů jazykového modelu.

Průběžná integrace (CI)

[cz] Vývojová praxe, při níž se změny často integrují a automaticky ověřují sestavením, testy a dalšími kontrolami, aby se integrační chyby odhalily co nejdříve.

R

Rozšíření

[cz] Rozšíření běžící přímo v prostředí harnessu, která rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické zachytné body.

S

Skript

[cz] Soubor nebo posloupnost příkazů určených k automatizovanému vykonání interpretem, shellem nebo jiným běhovým prostředím.

Sloučení commitů (Squash)

[cz] Operace, při níž se více po sobě jdoucích commitů nahradí jedním souhrnným commitem, obvykle za účelem zjednodušení historie před integrací změn.

Sloučení větví (Merge)

[cz] Operace správy verzí, která kombinuje změny nebo historii dvou vývojových linií do společného výsledného stavu; konflikty vyžadují explicitní vyřešení.

Smyčka ReAct (Agent Loop)

[cz] Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí.

Softwarové inženýrství

[cz] Systematické uplatňování inženýrských principů na specifikaci, návrh, implementaci, ověřování, provoz a údržbu softwarových systémů.

Softwarový kontejner (Container)

[cz] Izolované uživatelské běhové prostředí balící aplikaci a její závislosti při sdílení jádra hostitelského operačního systému; úroveň bezpečnostní izolace závisí na konkrétní implementaci a konfiguraci.

Správa verzí

[cz] Správa a sledování změn zdrojových souborů a dalších verzovaných artefaktů tak, aby bylo možné změny bezpečně větvit, slučovat, auditovat a v případě potřeby vracet.

T

Tah interakce (Turn)

[cz] Jedna diskrétní jednotka interakce v konverzačním nebo agentním protokolu, například zpráva uživatele, odpověď modelu nebo samostatně evidovaný výsledek nástroje.

Token

[cz] Diskrétní jednotka zpracovávaná jazykovým modelem. Token odpovídá položce slovníku tokenizéru a je reprezentován číselným identifikátorem; nemusí odpovídat celému slovu.

Tokenizér

[cz] Komponenta, která převádí text nebo jiný vstup na posloupnost tokenů a jejich identifikátorů a podle podporovaného směru také provádí zpětnou dekodovací transformaci.

Transformerová architektura (Transformer)

[cz] Architektura neuronových sítí založená na mechanismu pozornosti, která modeluje vztahy mezi prvky sekvence a tvoří základ většiny současných velkých jazykových modelů.

U

Událostní zachytný bod (Hook)

[cz] Definovaný bod životního cyklu nebo události, na který lze navázat vlastní deterministickou logiku před, po nebo místo standardního chování systému.

V

Vektorová reprezentace

[cz] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné sémantické vztahy mezi reprezentacemi.

Větev repozitáře (Branch)

[cz] Pojmenovaná vývojová linie v systému správy verzí, která umožňuje provádět změny odděleně od jiné linie historie a později je porovnat nebo sloučit.

Z

Zapojení člověka do smyčky (HITL)

[cz] Návrhový vzor, v němž lidský operátor zůstává součástí rozhodovacího procesu systému prostřednictvím schvalovacích bran (Human Gates), zejména před významnými nebo nevratnými systémovými operacemi.

Ř

Řídicí systém (Harness)

[cz] Řídicí postroj — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely, práci se stavem a deterministické řízení životního cyklu požadavku.

Seznam příloh

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
3. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
4. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
5. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
6. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
9. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
10. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
11. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>